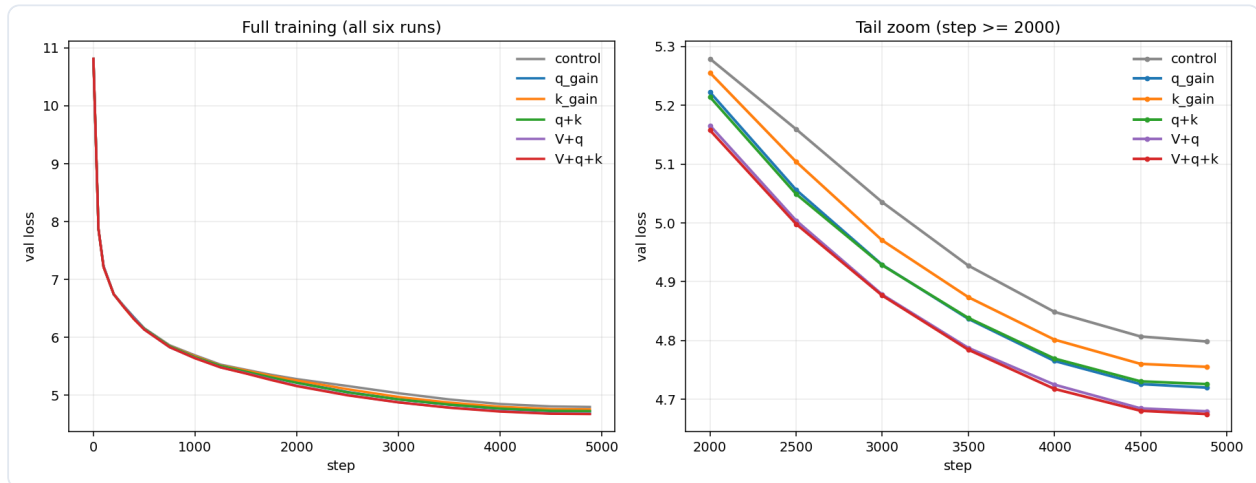


Let attention control its sharpness

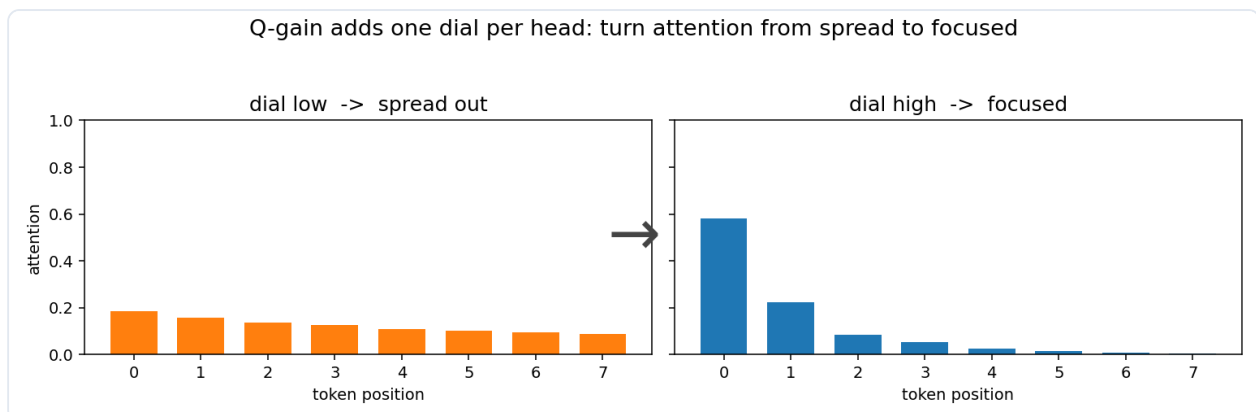
Add one learnable parameter per attention head that controls how diffused or focused attention scores are.

The curves below are from the `screen10m` setup: a ~7.7M parameter LLM trained on 20M tokens. Q-gain adds only 144 scalars on this model.



`g` is just a single learnable parameter that controls attention sharpness. One `g` per head.

It can be applied to queries (Q gain), keys (K gain), etc.



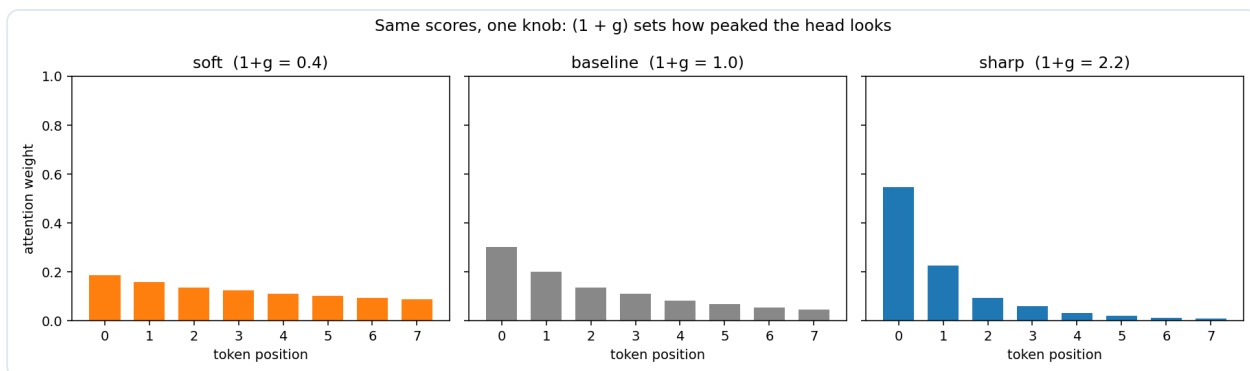
Standard attention baseline:

```
score = (Q @ K.T) / sqrt(d_head)
```

Q-gain scales that head's query vector before the score is computed:

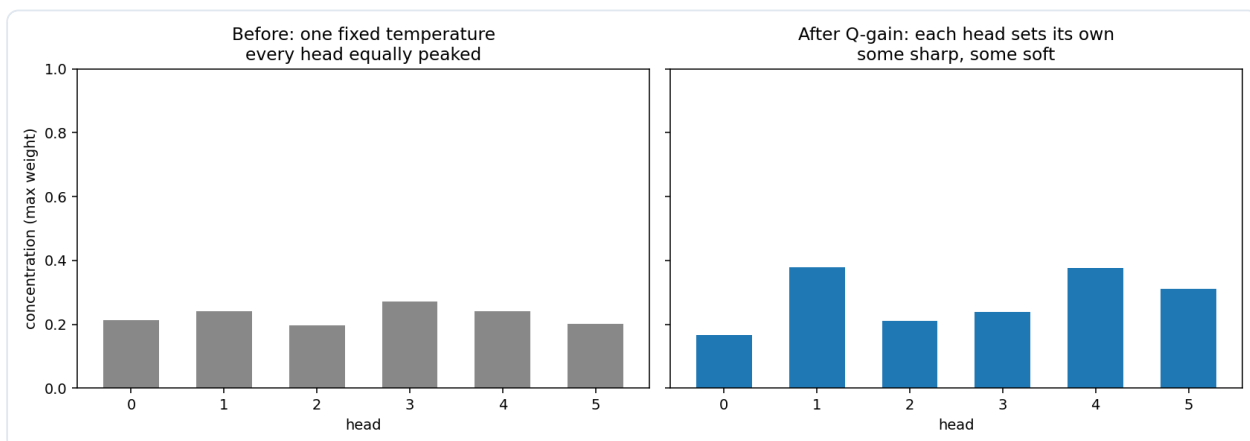
```
Q' = Q * (1 + g)
score' = (Q' @ K.T) / sqrt(d_head)
        = score * (1 + g)
```

So `g` is a direct sharpness knob on the softmax logits. We use `(1 + g)` so `g = 0` means "do nothing" and training starts as the exact baseline.



```
g < 0 -> flat    -> averages many tokens
g = 0 -> baseline (the dial starts here, changing nothing)
g > 0 -> peaked -> commits to one token
```

Every head picks its own. The baseline pins all heads to one sharpness. Q-gain lets each head choose - some sharp, some soft.



The code

One learnable scalar per head, initialized at zero:

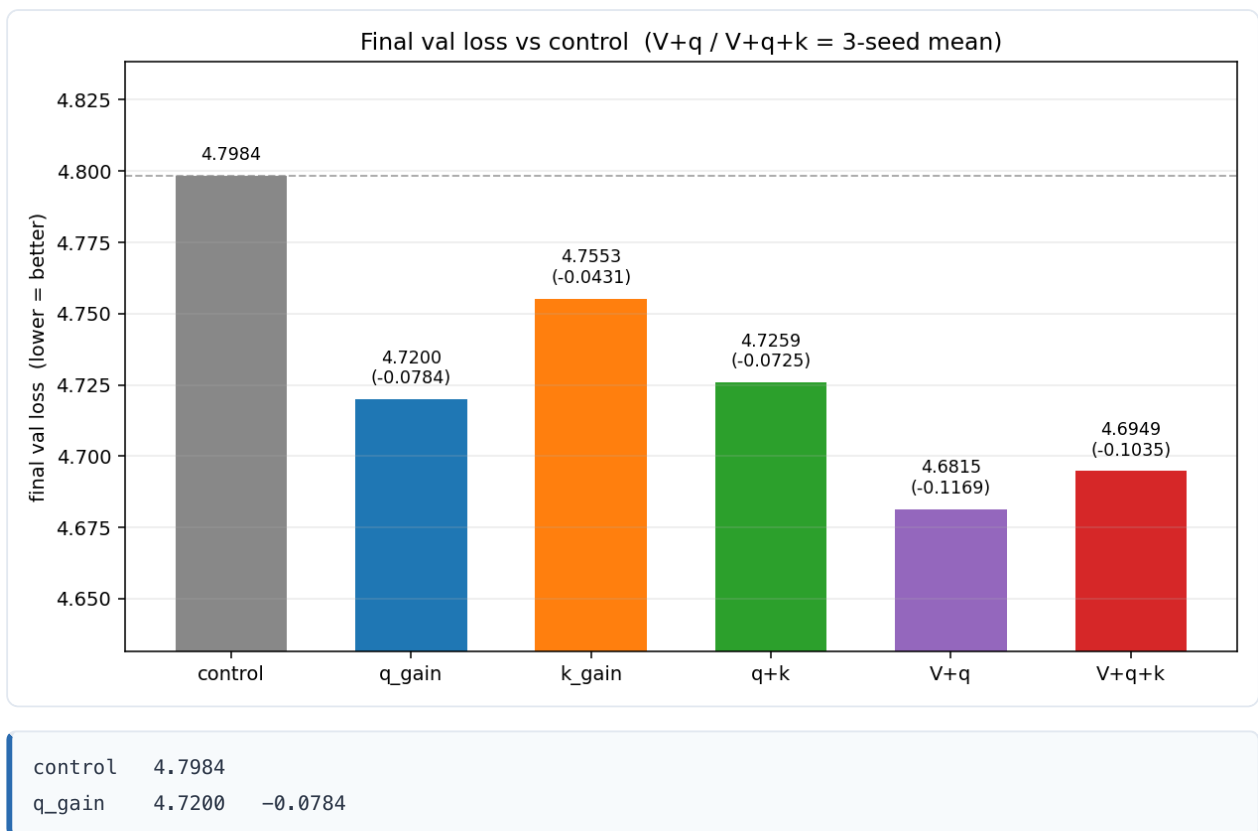
```
self.q_gain = nn.Parameter(torch.zeros(self.n_heads))    # one per head

Q_head = Q_head * (1.0 + g_head)
```

Cost on this [screen10m](#) model: 144 parameters (6 heads $\times$ 24 layers), zero new matmuls. You are not adding capacity - you are giving existing knobs a place to turn.

The result

Same config, seed 42, trained to 20M tokens:



-0.078 validation loss for 144 parameters. That is the lesson.